ML Model Deployment Strategies and Model Serving using FastAPI

College assignment report | 2026-10-23

Objective and dataset

This project trains and serves a passenger survival classifier using only the official Kaggle Titanic competition labelled train.csv (891 records). Kaggle test.csv was not used.

Preprocessing and model

The selected inputs are Pclass, Sex, Age, SibSp, Parch, Fare, and Embarked. Numeric missing values are imputed with the training-set median; categorical missing values use the training-set most-frequent value, followed by one-hot encoding. A ColumnTransformer and Pipeline keep preprocessing with a 300-tree RandomForestClassifier. The complete fitted pipeline is saved as model.pkl using Joblib. Preprocessing is fitted only on the training portion.

Training and evaluation

A stratified 80:20 split with random_state=42 produced 712 training and 179 held-out records. Measured accuracy was 82.1229% (147/179 correct). The held-out feature rows were exported without survival labels for unseen-record inference.

Inference strategies

Online inference predicts one held-out passenger per call. Batch inference sends 20 different held-out passengers through one prediction call and writes batch_predictions.csv. Streaming simulates ten separate incoming passenger events, predicting each individually with a one-second pause between events. The three subsets are disjoint.

FastAPI serving and model reuse

FastAPI provides a lightweight HTTP interface, typed Pydantic request validation, automatic OpenAPI documentation, and a health endpoint. The application loads model.pkl once at startup. Every inference mode uses the same fitted artifact, keeping preprocessing and predictions consistent and preventing retraining or data leakage during serving.

API validation and test results

Check	Observed status	Expected
GET /health	200	200
POST /predict (valid request)	200	200
POST /predict (Pclass = 4)	422	422
GET /docs	200	200
Python client	200	200

Additional checks: optional Age/Fare/Embarked request returned HTTP 200; invalid Age, Sex, and SibSp values each returned HTTP 422. The client returned HTTP 200.

Conclusion

The assignment demonstrates a leakage-aware train/test workflow and three deployment patterns backed by one persisted pipeline. The FastAPI endpoint validates passenger input and returns a class, readable label, and class probabilities.